Pipeline de Dados com IoT e Docker

Relatório acadêmico final

Aluno: Victor Matos

Disciplina: Disruptive Architectures: IoT, Big Data e IA

Aplicação: IoT Thermal Analytics | Victor Matos

Data da versão revisada: 14 de outubro de 2026

Apresentação

Este trabalho apresenta a construção de um pipeline para organizar e analisar leituras de temperatura de um conjunto de dados IoT. A solução utiliza Python para processamento, PostgreSQL em Docker para armazenamento e Streamlit para visualização dos resultados.

O relatório descreve as etapas de implementação, as três análises em SQL, os testes realizados, os principais resultados e as limitações dos dados. As telas da aplicação são apresentadas nos anexos.

Resumo

Este projeto implementa um pipeline de leituras reais de temperatura obtidas no dataset Temperature Readings: IoT Devices, disponibilizado no Kaggle por atulanandjha. O processamento em Python utiliza pandas, SQLAlchemy e psycopg2-binary; o armazenamento ocorre no PostgreSQL 16 executado em Docker; as análises são expostas por três views SQL e apresentadas em Streamlit com Plotly. O arquivo original contém 97.606 registros. A carga preserva todas as linhas no staging, identifica uma duplicata exata e mantém 97.605 leituras únicas na camada tratada. Três execuções registradas demonstram idempotência da tabela tratada. A documentação registra testes de ingestão, nove testes de leitura das views e do dashboard e dezesseis verificações do conteúdo das quatro exportações CSV. O relatório apresenta arquitetura, regras de qualidade, resultados, evidências visuais e limitações. Não se infere unidade térmica, fuso horário, sensores físicos ou disponibilidade operacional a partir de campos que não permitem essas conclusões.

1. Contextualização e objetivos

Leituras associadas a aplicações IoT podem chegar com frequências irregulares, registros repetidos e formatos inadequados à análise direta. Transformar um CSV em gráficos sem preservar sua origem dificulta explicar exclusões, reprocessar cargas e comparar resultados. O problema tratado neste trabalho é construir uma cadeia verificável entre a fonte, o processamento, o banco e a visualização.

O objetivo geral é processar leituras de temperatura, armazená-las em PostgreSQL com Docker e disponibilizar análises interativas baseadas exclusivamente em registros reais. Os objetivos específicos são preservar o arquivo de origem; validar formatos explicitamente; separar dados brutos, tratados e auditoria; impedir duplicação em reexecuções; produzir três perspectivas analíticas distintas; e documentar o significado e os limites das métricas.

Embora o dataset seja identificado como IoT, a implementação recebe um arquivo histórico em lote. Não há captura ao vivo de dispositivos, broker MQTT ou integração com hardware. A disciplina contempla IoT, Big Data e IA, mas esta entrega não implementa aprendizado de máquina nem afirma processamento distribuído. Seu foco é a engenharia de ingestão, armazenamento e análise descritiva exigida no desafio.

3. Arquitetura e organização

O fluxo do projeto começa no arquivo CSV original. O programa em Python lê os registros, verifica os formatos e salva as informações no PostgreSQL. Os dados brutos são preservados em uma área de staging; os dados tratados ficam em uma tabela própria, e os registros de auditoria permitem acompanhar cada execução.

As três views SQL produzem resumos diários, indicadores de distribuição das leituras por hora e comparações entre ambientes internos e externos. O dashboard consulta essas views e apresenta gráficos, indicadores, filtros e tabelas para exportação.

Principais componentes

Fonte: data/IOT-temp.csv.
Ingestão: src/ingest.py.
Banco: docker-compose.yml, com PostgreSQL 16.
Análises: sql/views.sql.
Consultas: src/dashboard_data.py.
Interface: dashboard.py, com Streamlit e Plotly.
Testes: pasta tests/.
Evidências: pasta docs/ e docs/screenshots/.

Ambiente de desenvolvimento

O projeto foi executado com Python 3.12.10, PostgreSQL 16.15 e Docker. Entre as principais bibliotecas utilizadas estão pandas, SQLAlchemy, psycopg2-binary, Streamlit e Plotly. O PostgreSQL é executado no contêiner, enquanto a ingestão e a interface são executadas no ambiente Python local.

4. Ambiente, Docker e proteção da configuração

O serviço postgres usa a imagem postgres:16. O Compose recebe as variáveis do arquivo .env local, exige senha definida e monta o volume postgres_data em /var/lib/postgresql/data. O contêiner deste projeto foi identificado nas validações anteriores como projeto_pipeline_vicmatos-postgres-1. Não foi recriado nesta etapa.

A porta interna é 5432. A publicação local utiliza 127.0.0.1 e POSTGRES_PORT, cujo padrão é 5432. Isso permite conexão das aplicações locais sem uma publicação em todas as interfaces de rede. O healthcheck usa pg_isready com usuário e banco definidos por variáveis; intervalo de 10 segundos, timeout de 5 segundos, cinco tentativas e período inicial de 10 segundos. Uma consulta SELECT version() confirmou PostgreSQL 16.15 na validação anterior. O resultado do healthcheck indica prontidão do banco, mas não comprova por si só a presença das views ou a qualidade dos dados.

Credenciais de conexão são lidas exclusivamente do .env. Esse arquivo e o ambiente .venv estão protegidos pelo .gitignore nas verificações registradas. .env.example documenta a configuração sem senha utilizável. SQLAlchemy recebe parâmetros de conexão sem imprimir a senha, e as mensagens de falha do dashboard não mostram o conteúdo sensível da exceção. Este relatório não incorpora .env nem credenciais.

Comandos de reprodução, a executar somente quando necessário e no projeto correto:

```
docker --context desktop-linux compose config --quiet
docker --context desktop-linux compose up -d postgres
docker --context desktop-linux compose ps
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m src.ingest
.\.venv\Scripts\python.exe -m streamlit run dashboard.py --server.port 8501
```

Esses comandos estão documentados para reprodução; não foram executados para gerar o relatório. Reexecutar ingestão mantém a tabela tratada idempotente, mas acrescenta staging e auditoria. Não se deve remover o volume para retomar a aplicação. O endereço http://127.0.0.1:8501 é local e não substitui um link público de entrega.

5. Dataset e qualidade dos dados

Fonte oficial: https://www.kaggle.com/datasets/atulanandjha/temperature-readings-iot-devices. Arquivo utilizado: data/IOT-temp.csv. O intervalo observado é de 28/07/2018 a 08/12/2018. O identificador de local é Room Admin e as categorias de ambiente são In e Out. O campo de local não identifica sensores físicos individuais.

Campo original	Preservação no staging	Campo tratado e tipo
id	Texto original	source_id TEXT, chave primária
room_id/id	Texto original, nome entre aspas no SQL	location_name TEXT
noted_date	Texto original	observed_at TIMESTAMP WITHOUT TIME ZONE
temp	Texto original	temperature SMALLINT
out/in	Texto original, nome entre aspas no SQL	environment TEXT, In/Out

O conjunto possui 97.606 linhas de dados e 97.605 identificadores únicos, com uma duplicata exata. A validação registrada não deixou rejeições persistentes para o arquivo oficial. Isso demonstra aprovação nas regras implementadas, não prova que toda leitura represente uma medição fisicamente correta. O CSV não é editado para corrigir formatos ou remover registros.

O SHA-256 do arquivo é 2bf965701db96cd5788403d395a25d2dd7282bea24e317b7f7b157b7a02063d4. A ingestão calcula esse hash e compara com o valor esperado antes de carregar. Uma alteração do arquivo resulta em recusa da carga oficial. O hash identifica o arquivo auditado, mas não demonstra, isoladamente, veracidade física da fonte.

As datas exigem o padrão DD-MM-AAAA HH:MM, validado por expressão regular e datetime.strptime com %d-%m-%Y %H:%M. Temperaturas exigem representação inteira e intervalo de SMALLINT, de -32768 a 32767; esse intervalo é uma restrição de armazenamento, não um limite físico de operação. Campos ausentes ou vazios e ambientes fora de In/Out são rejeitados. Unidade térmica e fuso não foram confirmados documentalmente.

6. Ingestão transacional, auditoria e idempotência

A carga lê o CSV com pandas em modo textual, sem converter silenciosamente valores ausentes. Os nomes e valores originais são preservados em staging.temperature_raw. Cada linha recebe source_row correspondente à sua posição no arquivo, começando em 2 por causa do cabeçalho, e run_id para vincular a execução.

Tabela	Finalidade
audit.ingestion_runs	Hash, arquivo, horários, status e contagens da carga
audit.ingestion_rejections	Linha rejeitada e motivo, vinculados ao staging
staging.temperature_raw	Valores originais de todas as linhas de cada execução
iot.temperature_readings	Leituras tipadas, ID único, local, data, temperatura e ambiente

Um bloqueio consultivo transacional serializa as cargas. A inserção em lotes com psycopg2 participa da transação controlada pelo SQLAlchemy. Criação das estruturas, staging, rejeições, registros tratados e conclusão da auditoria são confirmados juntos. Erros técnicos revertem toda a transação; consequentemente, uma carga que falha tecnicamente não deixa seu registro de falha persistido nesta versão.

Duplicatas são detectadas pela igualdade dos cinco valores originais. A ocorrência adicional permanece no staging e é contabilizada como duplicata, sem nova leitura tratada. Observações simultâneas com IDs diferentes são preservadas. Variantes distintas de um mesmo ID no arquivo são rejeitadas, e um ID existente no banco só é considerado já carregado quando seu conteúdo tratado coincide. Conflitos recebem motivo auditável, em vez de sobrescrever medições.

A reconciliação exigida é: origem = duplicatas + rejeições + inseridos + já existentes. A aplicação valida a igualdade e a tabela de auditoria exige essa relação para status completed. A chave primária source_id impede duplicação da camada tratada. Staging e auditoria crescem a cada execução bem-sucedida, mesmo que nenhuma leitura nova seja inserida; idempotência refere-se à tabela tratada.

Execução registrada	Origem	Duplicatas	Rejeições	Inseridos	Já existentes
Primeira carga	97.606	1	0	97.605	0
Segunda carga	97.606	1	0	0	97.605
Terceira carga	97.606	1	0	0	97.605
Total auditado	292.818	3	0	97.605	195.210

O registro final_integrity_check.json documenta três execuções concluídas, 292.818 linhas em staging, zero rejeições persistentes e 97.605 IDs únicos. As três ocorrências de duplicata representam a mesma duplicata contabilizada em três cargas, e não três duplicatas diferentes na fonte. O fingerprint de conteúdo db628f44918517e86003d1e23d9bcb7b permaneceu igual nas verificações registradas. Esse MD5 serve à comparação de integridade entre etapas; não é uma assinatura de segurança.

7. Três views SQL analíticas

7.1 Resumo diário: vw_daily_temperature_summary

A view agrupa observed_at convertido em data, location_name e environment. Retorna temperatura mínima, máxima, média, amplitude e reading_count. A amplitude é máxima menos mínima. A média é calculada sobre as leituras do grupo; não representa ponderação por duração. O resultado validado possui 158 grupos. As contagens somadas reconciliam 97.605 leituras.

Exemplo real: em 28/07/2018, Room Admin, In, foram registradas 45 leituras, todas com temperatura 31. Mínimo, máximo e média são 31; amplitude é zero. Esse resultado descreve os registros disponíveis daquele dia e ambiente, sem concluir estabilidade contínua durante as horas sem observações.

Uso analítico: comparar variação diária entre ambientes, localizar dias de maior amplitude e contextualizar extremos com a quantidade de leituras. Ao resumir vários grupos, a média correta é a soma de média do grupo vezes contagem, dividida pela soma das contagens. Uma média simples das médias daria o mesmo peso a grupos de volumes diferentes.

7.2 Cobertura: vw_hourly_sampling_coverage

A view agrupa data, hora inteira de 0 a 23, local e ambiente. Retorna reading_count, observed_minutes, minute_coverage_pct, has_unobserved_minutes e additional_readings_in_observed_minutes. A contagem de minutos usa timestamps truncados ao minuto e distintos dentro do grupo. A cobertura é observed_minutes × 100 / 60. O resultado validado possui 2.287 grupos, cuja soma das leituras também é 97.605.

Exemplo real: em 28/07/2018 às 07h, Room Admin, In, há 45 leituras em 7 minutos distintos. A cobertura é aproximadamente 11,6667%, e há 38 leituras além da primeira em minutos já observados. Elas não são classificadas como duplicatas por compartilharem minuto. Há 2.265 grupos com menos de 60 minutos observados e 717 com leituras adicionais em minutos observados.

A view só retorna horas com registros. Não cria horários ausentes nem preenche medições. O denominador é sempre 60, inclusive em horas parciais nas extremidades do arquivo. A métrica descreve distribuição temporal dos registros, não disponibilidade comprovada do sensor. Pode orientar uma investigação da coleta, mas não diagnosticar indisponibilidade de hardware.

7.3 Comparação: vw_hourly_indoor_outdoor_gap

A view agrupa timestamp truncado à hora e local. As médias de In e Out são calculadas separadamente com agregações filtradas e acompanhadas de suas contagens. HAVING mantém somente horas com registros nos dois ambientes. O resultado contém 865 grupos comparáveis. Não há JOIN entre leituras brutas; assim, a comparação não multiplica registros por combinações entre ambientes.

A convenção é outdoor_minus_indoor = média externa − média interna. Um resultado positivo indica média externa maior. Em 28/07/2018 às 07h, Room Admin, a média interna é 31, com 45 leituras; a média externa é 31,7906976744186047, com 43 leituras. A diferença é 0,7906976744186047. Os valores resultam de somas 1.395 e 1.367 divididas pelas respectivas contagens.

As médias são ponderadas por leituras em cada ambiente. Estar na mesma hora não garante simultaneidade dos registros, e frequências distintas podem influenciar a comparação. Horas com apenas um ambiente são excluídas. A diferença não demonstra causalidade ou eficiência térmica de uma edificação. As três views utilizam CREATE OR REPLACE VIEW compatível e deixam a ordenação para as consultas consumidoras.

8. Dashboard e interpretação dos indicadores

A aplicação IoT Thermal Analytics | Victor Matos possui cinco páginas: Visão geral, Temperaturas diárias, Cobertura das leituras, Comparação In/Out e Qualidade e limitações dos dados. A interface foi desenvolvida em Streamlit e os gráficos utilizam Plotly.

Os filtros permitem selecionar um período de datas e os ambientes In e Out. Na página de comparação, são necessárias leituras dos dois ambientes para calcular a diferença. Quando não existem registros para os filtros escolhidos, a aplicação apresenta mensagens explicativas, sem criar valores inexistentes.

Indicadores apresentados

No período completo, o dashboard apresenta 97.605 leituras, temperatura mínima de 21, máxima de 51 e média de aproximadamente 35,0539. A média é ponderada pela quantidade de leituras. Como a unidade não foi confirmada na fonte, os valores não são identificados como graus Celsius.

A página de cobertura mostra em quantos minutos de cada hora existem observações. Esse indicador ajuda a perceber intervalos com poucos registros, mas não permite afirmar que o sensor ficou desligado. A comparação In/Out mostra a diferença entre médias dos dois ambientes nas horas em que ambos possuem registros.

Interação e exportação

As páginas analíticas disponibilizam tabelas e exportação em CSV. As consultas são realizadas em modo somente leitura e há atualização dos dados por meio da interface. As capturas das cinco páginas e dos gráficos estão reunidas no anexo B.

9. Testes e validação

Foram realizados testes para conferir a ingestão, os cálculos das views e o funcionamento das consultas do dashboard. A validação da ingestão incluiu tratamento de duplicatas, reconciliação das contagens e comportamento em situações inválidas.

As views foram comparadas com cálculos independentes sobre os registros do banco. O dashboard foi verificado quanto aos indicadores, filtros e situações sem resultados. A suíte final de leitura teve nove testes aprovados, sem falhas.

Exportações CSV

Também foram realizadas 16 verificações automatizadas das quatro exportações CSV, avaliando nomes, cabeçalhos, codificação, contagens e correspondência com as consultas. No período completo, os arquivos de resumo diário e temperaturas diárias possuem 158 linhas cada; a cobertura horária possui 2.287 e a comparação horária possui 865.

Com o filtro de 01 a 10/08/2018, considerando In e Out, as contagens esperadas foram 3, 3, 7 e 3, respectivamente. Os testes automatizados foram aprovados, mas os botões de download não foram conferidos manualmente em uma sessão de navegador.

Evidências

O conjunto de evidências inclui as consultas de validação, os resultados dos testes e as capturas do dashboard. A tabela tratada permaneceu com 97.605 registros únicos nas verificações de integridade.

10. Resultados, insights e uso prático

O primeiro resultado é a preservação de 97.605 leituras únicas, com uma duplicata exata explicada pela auditoria. Reprocessamentos não aumentaram a tabela tratada. Isso permite reproduzir análises sem apagar dados e demonstra a utilidade de separar idempotência analítica de crescimento do histórico de carga.

O intervalo total de temperaturas registradas vai de 21 a 51, com média por leitura de aproximadamente 35,0539. A diferença entre extremos é 30 na unidade não confirmada. A amplitude global não deve ser atribuída a um único dia ou ambiente. Para examinar contextos locais, o resumo diário reúne extremos e volume em cada grupo.

A amostragem temporal é desigual: 2.265 dos 2.287 grupos horários têm menos de 60 minutos observados, aproximadamente 99,04%. Esse percentual descreve somente horas presentes na view, não a totalidade de horas do calendário nem indisponibilidade. Em 717 grupos há mais de uma leitura por minuto observado. Muitas leituras podem coexistir com baixa dispersão temporal, como as 45 observações em apenas 7 minutos do exemplo real. Portanto, volume de registros não substitui cobertura temporal.

As 865 horas comparáveis viabilizam examinar diferenças In/Out sem preencher horários ausentes. A amostra de 28/07/2018 às 07h mostra média externa maior em cerca de 0,7907, mas não permite generalizar esse sinal a todo o período nem concluir uma causa. É necessário analisar o gráfico e suas contagens antes de formular uma interpretação mais ampla.

Em um ambiente real, o resumo diário poderia apoiar triagem de dias com amplitude incomum; a cobertura poderia orientar revisão da frequência de coleta e comparação com logs de transmissão; e a diferença In/Out poderia apoiar investigação do comportamento térmico junto de dados de ocupação e climatização. Essas aplicações são propostas, não operações implementadas. Alertas exigiriam confirmação da unidade, limites operacionais, identificação dos sensores e definição da frequência esperada.

Os diferenciais desta solução são rastreabilidade por linha e execução, preservação textual do staging, reconciliação obrigatória, conflitos de ID tratados sem sobrescrita, comparação antes de qualquer combinação de registros e distinção explícita entre cobertura observacional e disponibilidade. A solução não depende de inferir equipamentos ou copiar modelos de outros alunos para produzir interpretações úteis.

11. Limitações e possibilidades de evolução

O dataset não identifica sensores físicos individuais. Room Admin representa local, e In/Out representa ambiente. Unidade térmica e fuso não estão confirmados: não se presume Celsius nem se convertem datas para UTC. As médias são ponderadas por registros e podem refletir coleta irregular; não equivalem a médias ponderadas pelo tempo.

Ausência de observações não distingue dispositivo desligado, falha de comunicação, coleta intermitente ou recorte do arquivo. O denominador de cobertura é 60 mesmo em horas parciais. Comparações horárias não garantem simultaneidade nem causalidade. Linhas entre pontos não demonstram medições realizadas no intervalo.

A implementação é local e em lote. Não há validação de escala Big Data, pipeline distribuído, modelo de IA, autenticação de dashboard ou hospedagem pública. A camada de staging cresce por execução, exigindo política de retenção futura. Falhas técnicas revertidas não deixam registro persistente de falha de ingestão; logging externo seguro é uma evolução possível. O cache pode exibir dados de até 60 segundos antes da atualização.

Evoluções possíveis incluem retenção documentada, trilha externa de falhas, ingestão incremental com contrato de dados, metadados confiáveis de sensores/unidade/fuso e indicadores de frequência esperada. Só depois desses fundamentos seria adequado avaliar alertas operacionais ou modelos preditivos. Nenhuma dessas melhorias foi declarada implementada.

12. Conclusão

O projeto alcançou o objetivo de construir um pipeline de dados com Python, Docker, PostgreSQL e Streamlit. A partir de um arquivo com 97.606 registros, foi identificada uma duplicata exata e formada uma tabela tratada com 97.605 leituras únicas. O processo também preserva os dados originais e mantém informações de auditoria.

As três views SQL oferecem formas diferentes de analisar os registros: resumo diário das temperaturas, distribuição das leituras por hora e comparação entre ambientes In e Out. No dashboard, essas análises podem ser consultadas por meio de gráficos, tabelas e filtros.

Os testes de leitura foram aprovados e as evidências foram documentadas. A conferência manual dos botões de download permanece como limitação da validação. Outra limitação importante é que a fonte não confirma a unidade de temperatura nem o fuso horário, o que impede interpretações mais específicas.

Como continuidade, o projeto poderia receber dados de sensores em tempo real, registrar metadados dos equipamentos e oferecer alertas com limites de temperatura definidos. Essas funcionalidades são possibilidades futuras, não partes já implementadas.

Disponibilização

O código, os testes, a documentação e este relatório estão disponíveis no repositório público: github.com/victorferreiramathos-star/pipeline-iot-docker-victor-matos. A apresentação em vídeo será entregue separadamente.

Referências

ATULANANDJHA. Temperature Readings: IoT Devices. Kaggle. Disponível em: https://www.kaggle.com/datasets/atulanandjha/temperature-readings-iot-devices.

Orientação acadêmica. Trabalho da disciplina Disruptive Architectures: IoT, Big Data e IA. Material disponibilizado para a atividade.

Arquivos do projeto utilizados

Implementação: src/ingest.py; src/dashboard_data.py; dashboard.py; sql/views.sql; docker-compose.yml.

Testes e validação: pasta tests/; docs/views_validation.md; docs/dashboard_validation.md; docs/final_evidence_tests.txt; docs/final_integrity_check.json; docs/visual_csv_validation.json; docs/final_readonly_validation.json.

Evidências visuais: docs/screenshots/, com capturas das cinco páginas do dashboard e imagens complementares.

Repositório público

https://github.com/victorferreiramathos-star/pipeline-iot-docker-victor-matos

Figura 1 — Visão geral: identificação, filtros e indicadores.

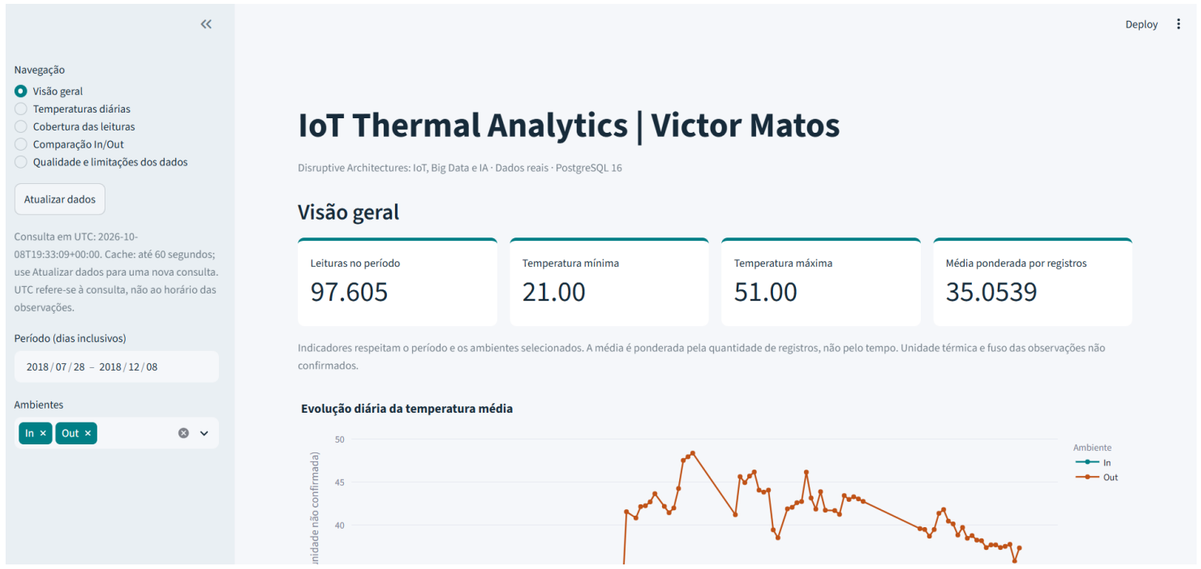


Figura 2 — Visão geral: evolução diária das médias disponíveis.

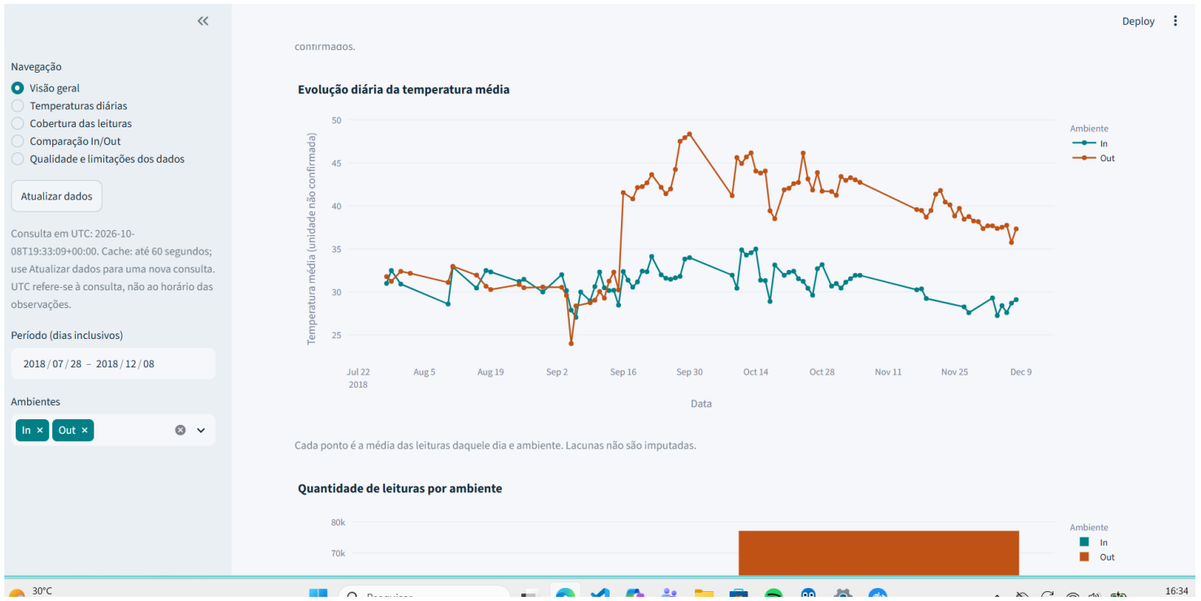


Figura 3 — Visão geral: quantidade de leituras por ambiente.

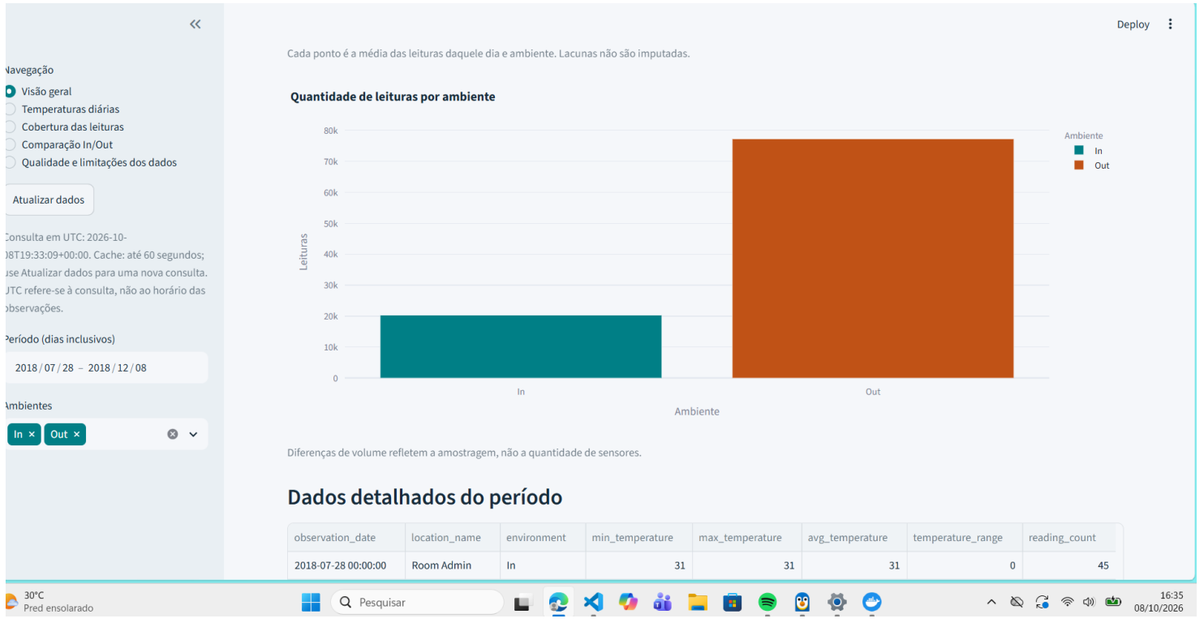


Figura 4 — Visão geral: tabela e controle de exportação CSV.

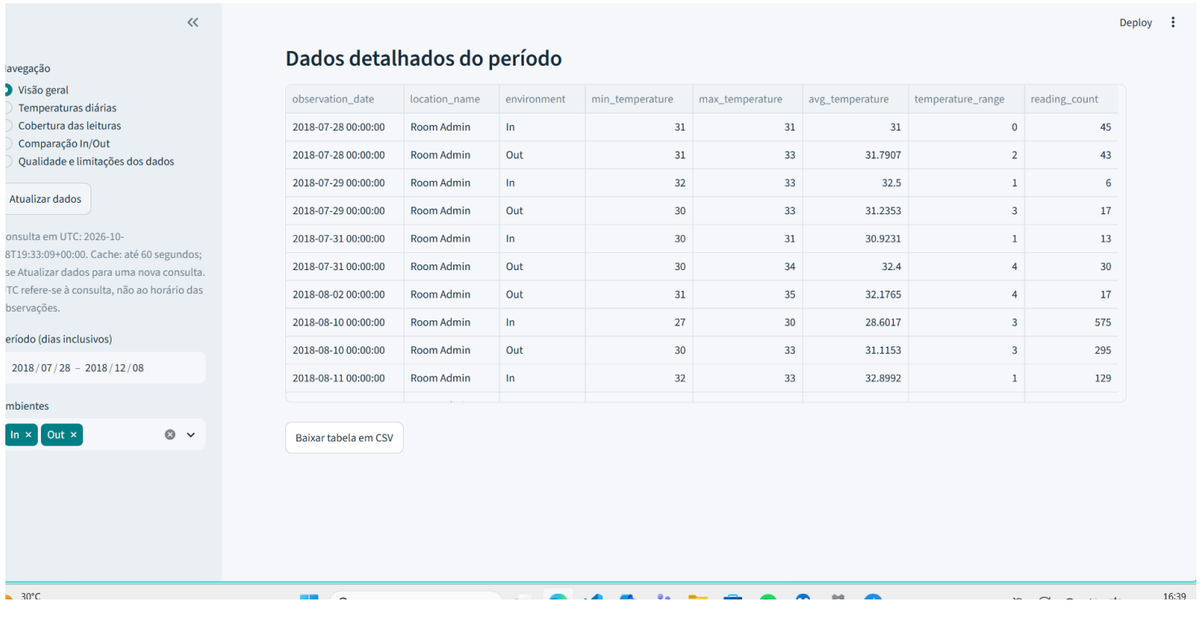


Figura 5 — Temperaturas diárias: identificação, filtros e indicadores.

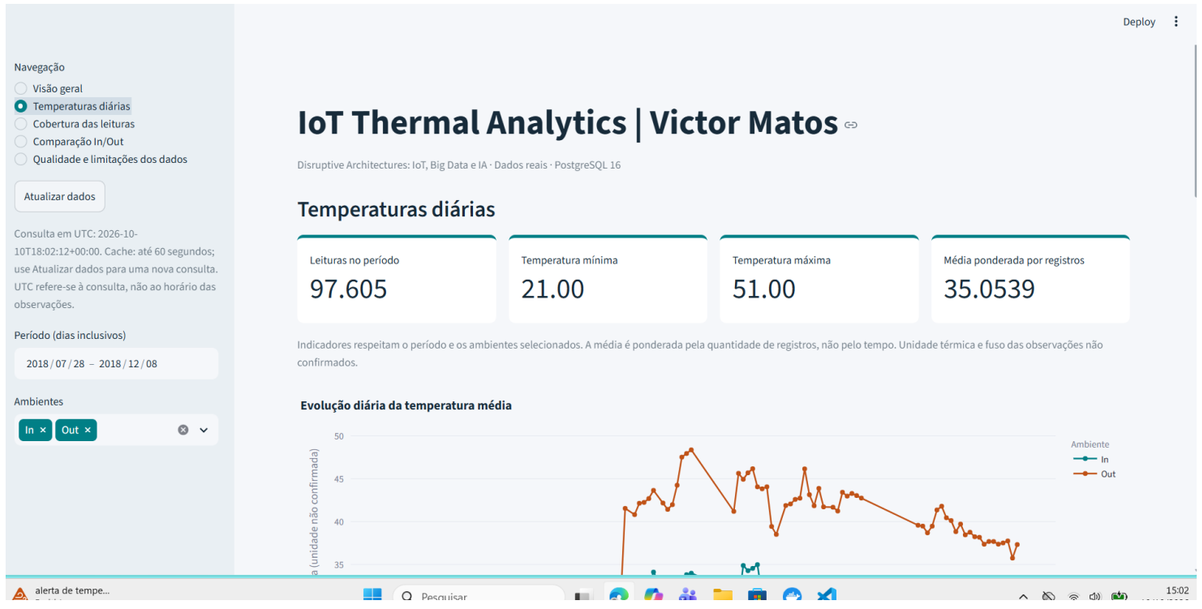


Figura 6 — Temperaturas diárias: mínimos e máximos.

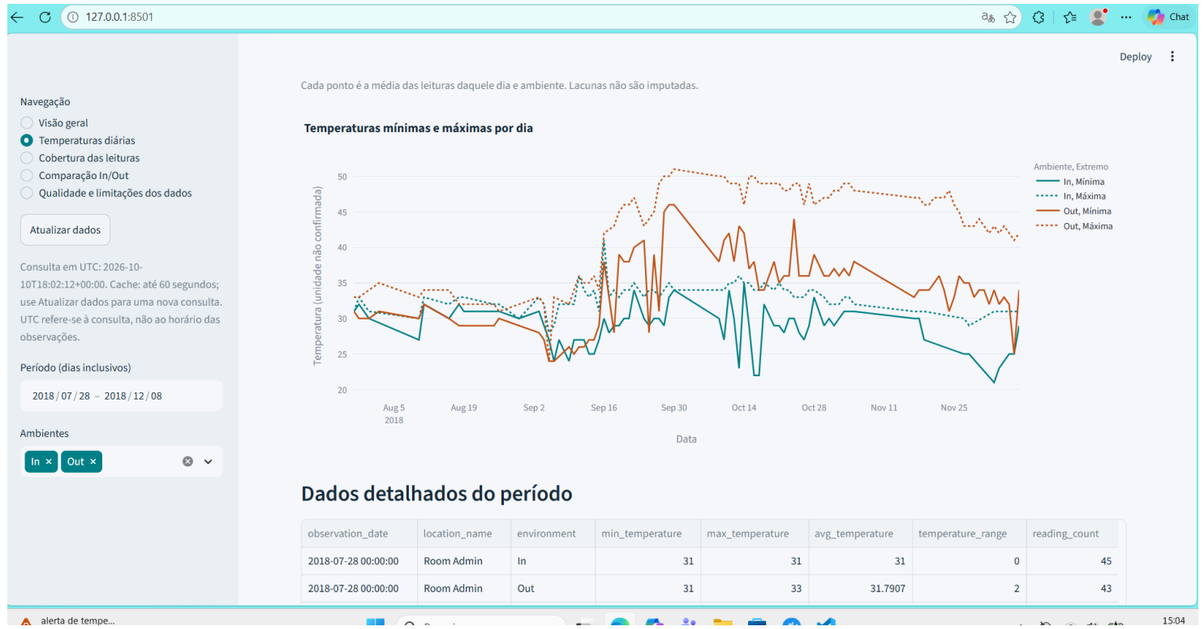


Figura 7 — Cobertura das leituras: identificação e gráfico.

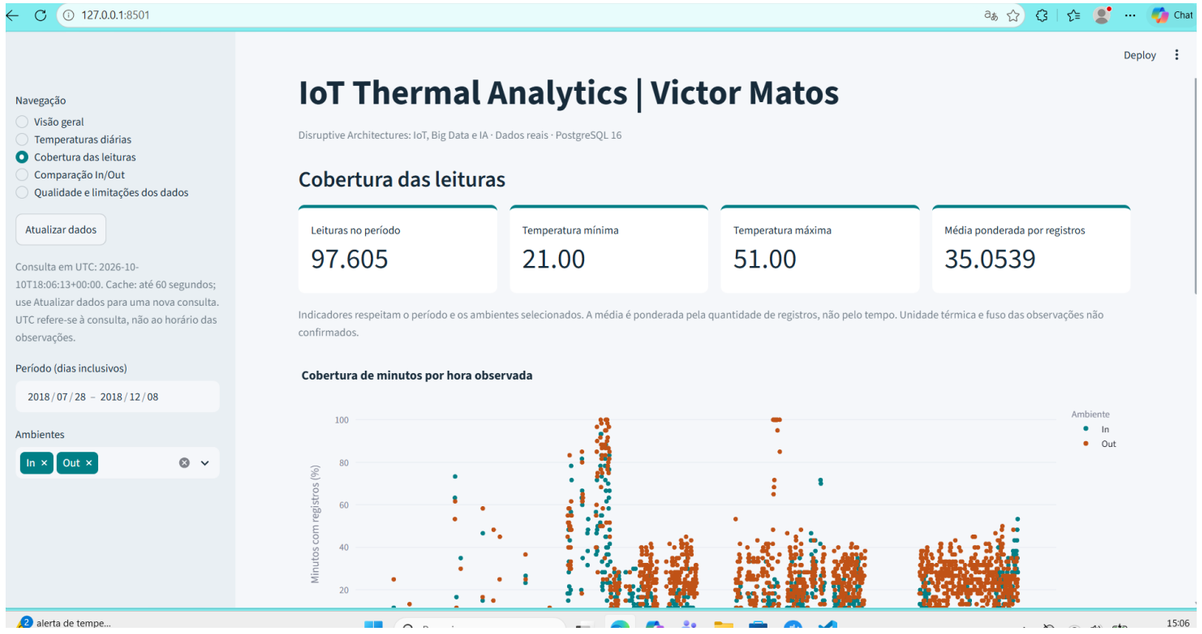


Figura 8 — Cobertura: aviso de interpretação, contagens e tabela.

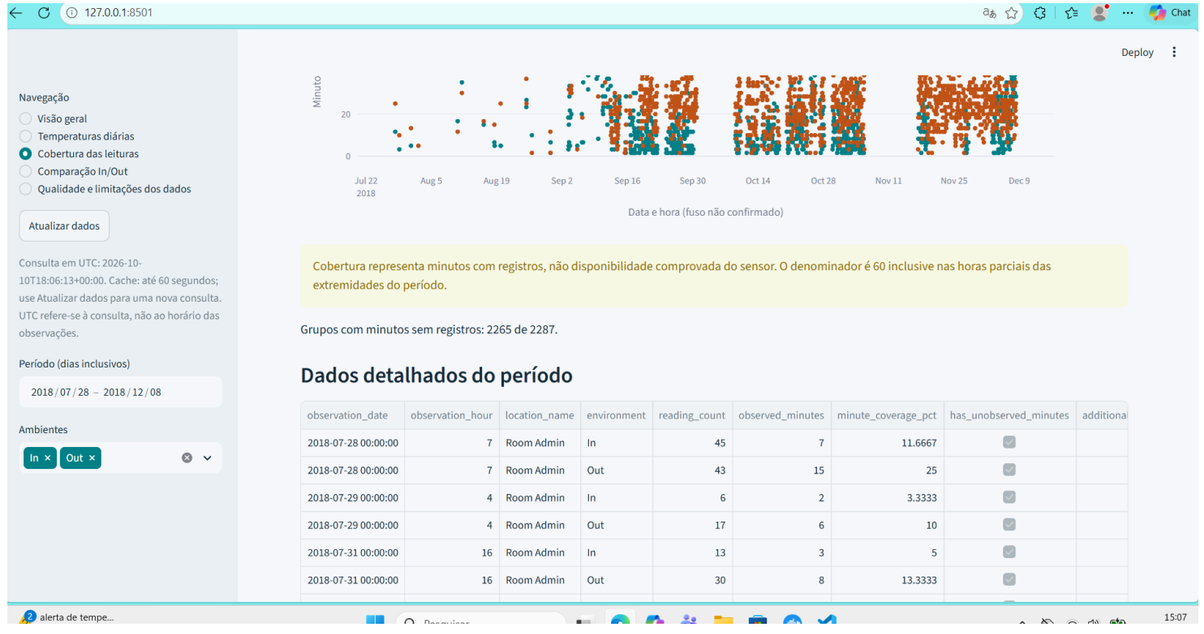


Figura 9 — Comparação In/Out: identificação, filtros e convenção da diferença.

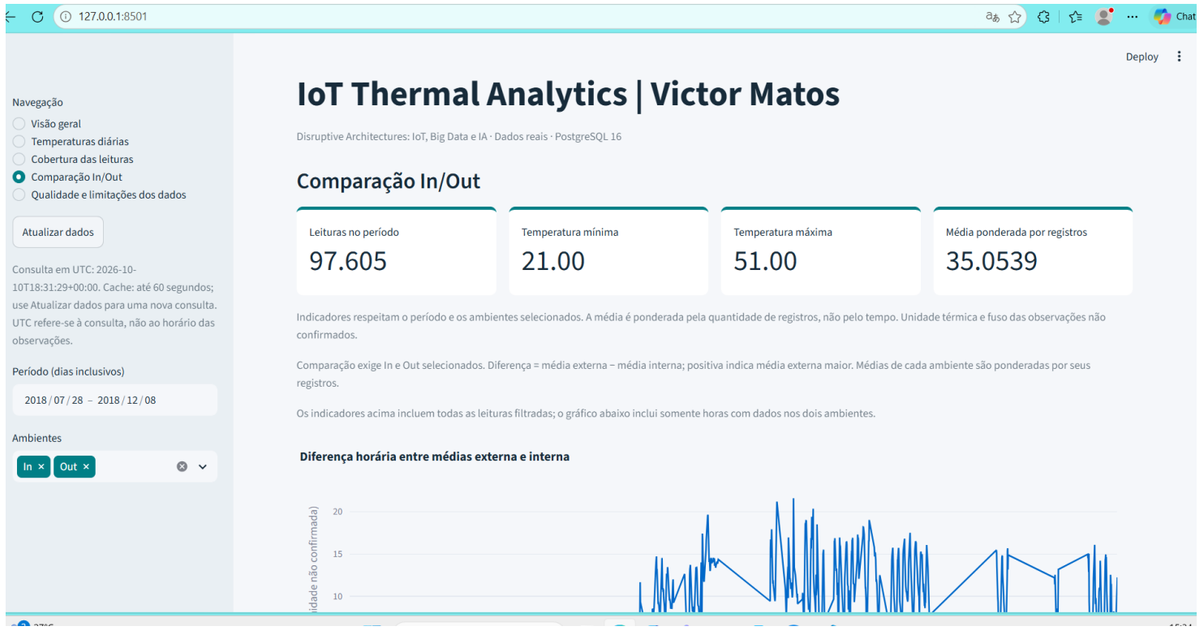


Figura 10 — Comparação In/Out: gráfico da diferença externa menos interna.

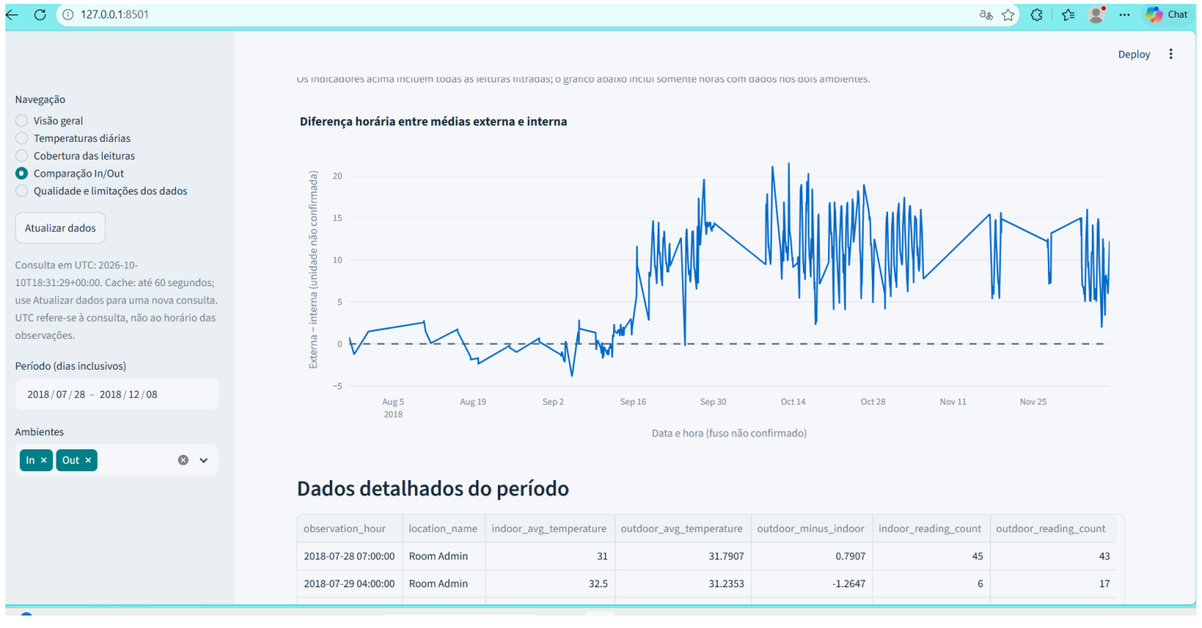


Figura 11 — Comparação In/Out: tabela e controle CSV.

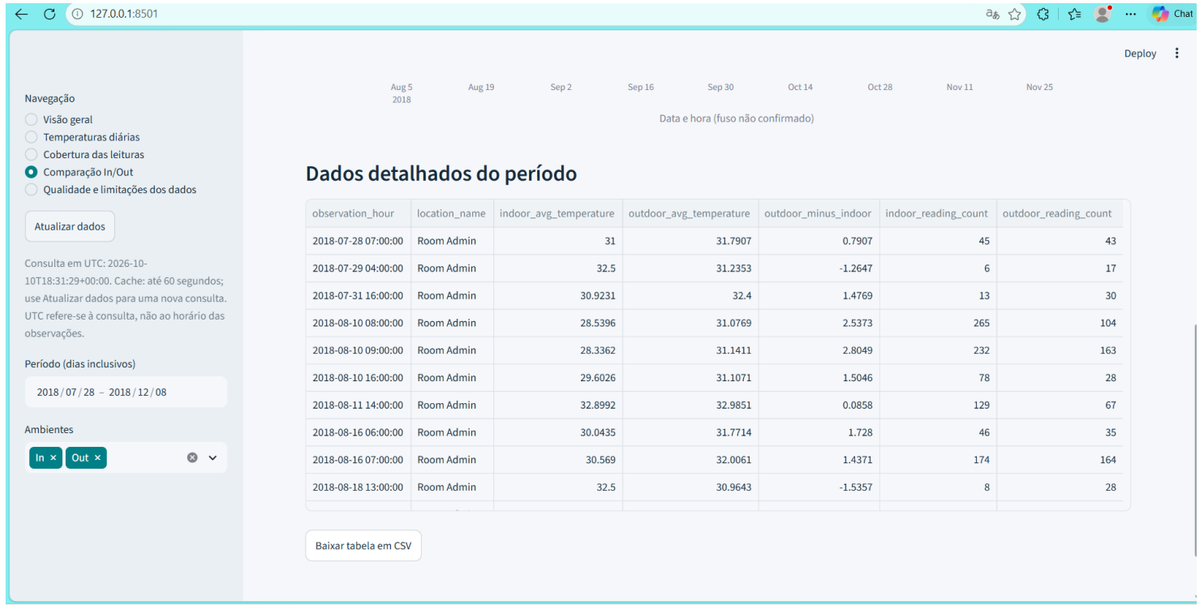


Figura 12 — Qualidade e limitações: identificação e contexto.

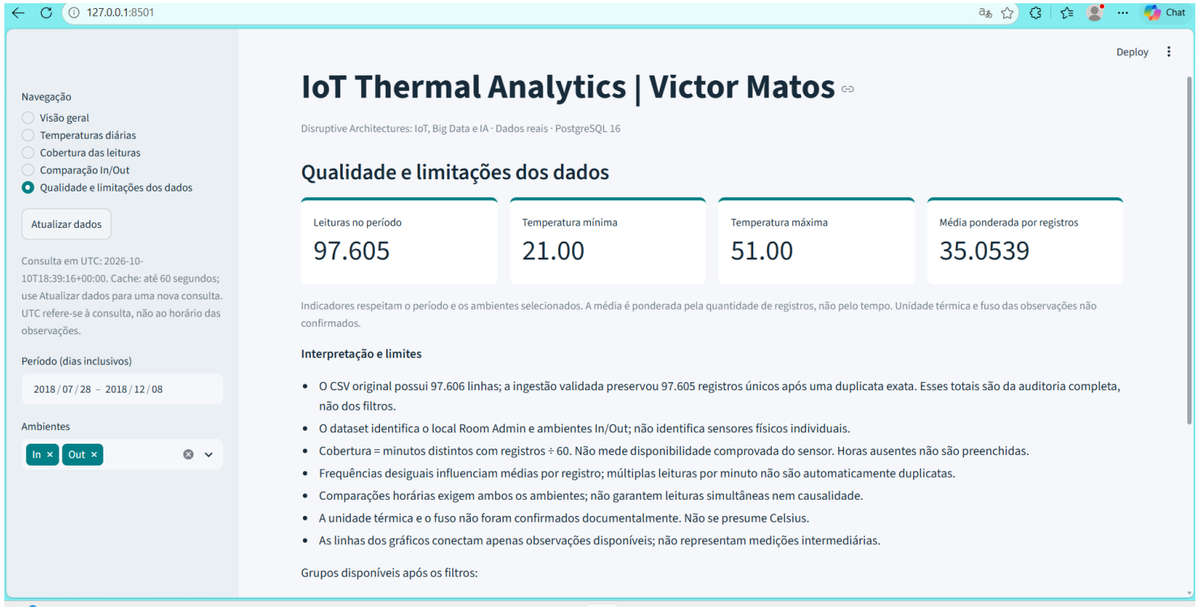


Figura 13 — Qualidade e limitações: avisos e contagens dos grupos.

